Module 8: Portfolio Project

Mukul Mondal

Colorado State University Global

MS - Artificial Intelligence and Machine Learning

CSC525 - Principles of Machine Learning

Dr. Dong Nguyen

July 1, 2026

Module 8: Portfolio Project

Problem statement: Option #2: NLP Chatbot Project Final Version

Please submit your final NLP chatbot project in the form of an executable file or link allowing the instructor to chat with your chatbot. chatbot must meet the following conditions:

- Use NLP learning methods to respond to user inputs.
- Chatbot responses should not be nonsense.
- Responses should be clearly in response to the input text.

Please include with your submission at least a page stating: whether the chatbot is open-domain or closed, what tools and libraries used, and what type of NLP model is used in the chatbot, as well as any instructions for running or accessing the chatbot.

My solution: I previously outlined in earlier project milestones the plan to use *Ollama* for developing the chatbot application. I have also built and tested several required components. In this phase of the project, I will integrate all existing components, perform comprehensive testing, and develop any additional modules needed to deliver a fully functional chatbot application.

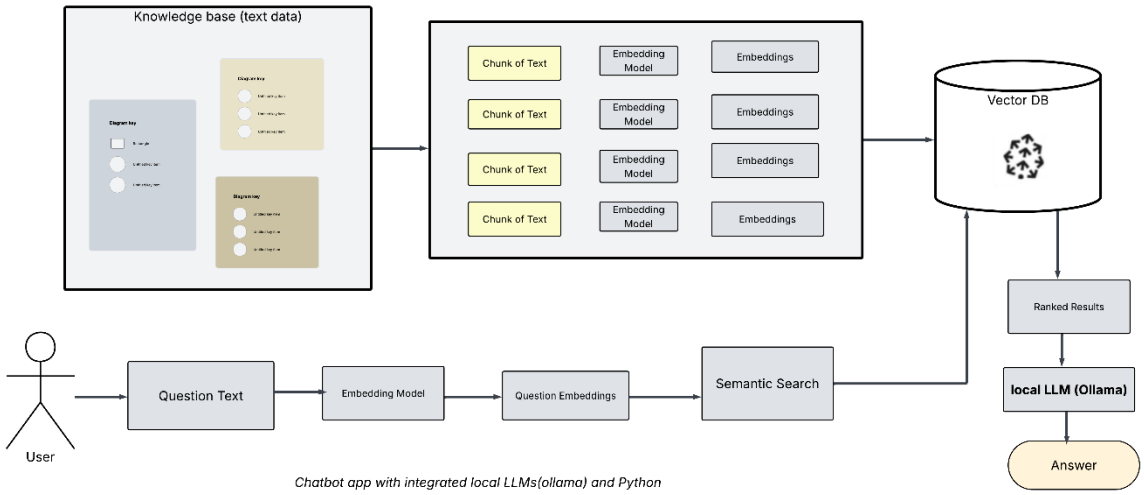
Basics of chatbot app using *Ollama, langgraph, vectordb*: A learning-based chatbot using *Ollama* integrates local large-language-model inference with adaptive Natural Language Processing techniques to deliver efficient, privacy-preserving automation. It begins by converting user input into vectorized semantic representations through embeddings generated by an *Ollama*-hosted model. These vectors support intent recognition, entity extraction, and retrieval of domain-specific knowledge. A RAG pipeline pairs Ollama with a vector store such as *ChromaDB*, enabling the chatbot to ground responses in accurate customer-service documentation. Learning occurs through continuous feedback loops: user interactions are logged, evaluated, and reintegrated into the knowledge base to refine retrieval quality and conversational

accuracy. *LangChain* builds on these capabilities by offering structured NLP workflows such as prompt templates, document preprocessing, and RAG retrieval through vector stores like *ChromaDB*. Because *Ollama* runs models locally, the system offers low-latency inference and strong data control. Combined with structured dialogue management, this architecture produces a scalable, adaptive customer-service assistant that improves over time.

Role of vector database: A vector database plays a central role in a chatbot built with *Ollama*, *LangGraph*, and *ChromaDB* because it enables fast, accurate, and semantically meaningful retrieval of information. In this architecture, *ChromaDB* stores high-dimensional vector embeddings generated from chunked documents. Each chunk is converted into a numerical representation that captures its semantic meaning. When a user submits a query, the system embeds the query using an embedding model running locally through *Ollama*. *LangGraph* then orchestrates the workflow by sending this query embedding to the vector database. *ChromaDB* performs a similarity search, comparing the query vector against stored vectors to identify the most relevant document chunks. This retrieval step is essential because it grounds the chatbot's responses in factual, domain-specific information rather than relying solely on the LLM's internal knowledge. The retrieved chunks are then injected into the prompt, enabling the model to produce accurate, context-aware answers. In short, the vector database acts as the chatbot's long-term memory, enabling efficient semantic search, reducing hallucinations, and ensuring that responses remain aligned with the user's data.

Adding data to *ChromaDB* is not training. It is indexing or storing embeddings, not updating or teaching the model. Training means updating model weights, changing how the model reasons, improving predictions, learning new patterns and this requires at least gradient descent, huge datasets. What ChromaDB does instead (vector storage): When we "add text data"

to ChromaDB, it converts the 'input text' to an embedding vector (using *Ollama* or another model) then store that vector in Chroma db. Later based on our query, it retrieves the closest vectors using similarity search. So, in a way, this is indexing, not training. Even though it's not training, ChromaDB is powerful because it enables: semantic search, context injection, domain-specific answers, long-term memory for the app. My chatbot app will need these components for building and executing the application.



Chatbot app with integrated local LLMs(ollama) and Python

Basic App Architecture Diagram

Tools, libraries, and models used: (these may change slightly based on my further study, research, and version availability, but basic constructs will remain same).

- *Ollama* (local LLM Runtime) -- *Ollama* runs large language models locally and provides LLM inference for models (*llama3.2*), Embedding models (e.g. *mxbai-embed-large*) otherwise model management and versioning. This is the engine that generates the chatbot's responses.
    - *Llama 3.2* is a small, efficient AI model designed to understand text, generate text, answer questions, reason, summarize, and assist with tasks. It trades raw

power for speed, low memory usage, and portability, making it ideal for developers building local chatbots, RAG systems, agents, and automation tools.

- *mxbai-embed-large* is an embedding model used to convert text into numerical vectors so that a system can search, compare, and retrieve information based on meaning rather than exact words. It transforms text (sentences, paragraphs, documents) into high-dimensional vectors that capture meaning, not just keywords. These embeddings are then used for Semantic search, RAG pipelines, Similarity matching, Clustering, Document retrieval. *mxbai-embed-large* is known for high accuracy in semantic similarity, Strong performance on retrieval benchmarks, Compatibility with vector databases like *ChromaDB*, Excellent pairing with small LLMs (like Llama 3.2) in RAG systems, good generalization across domains.

- *LangChain*: It performs the job of Retrieval-Augmented Generation (*RAG*), document ingesting and chunking, workflow orchestration etc.
- *ChromaDB*, the Vector Database -- *ChromaDB* stores and retrieves embeddings, otherwise performs fast similarity search, metadata filtering, persistent local database.
- Python Libraries etc: pip install *langchain langchain-core langchain-ollama langchain-chroma chromadb pandas requests*.

Input dataset: Input dataset information provided in the Module 4 project milestone, among these I've selected the data source: Customer Support Tweets on Twitter. I've downloaded the data file: *sample.csv* and added in my project. This appears to be a snapshot from Apple's iPhone customer support logs. The *['text']* column in the dataset will be the most significant for my work. Besides that, I will also try to use other columns such as *['tweet_id']* and

['created_at']. These are web-sourced data and will likely require some cleanup, such as removing emojis, HTML tags, and other extraneous elements. All these are done in module 4 milestone function: *def loadAndCleanupFileData(dataFile: str):* This function does all these and then returns a clean *pandas.dataframe*.

Vector database (ChromaDb) and embedding: For text embedding, I'm using the ollama model *mxbai-embed-large*. This is done in function: *def createEmbeddingInVectorDb(dfInput: pd.DataFrame, dbFile: str, collectionName: str):* Basic Implementation logic:

create embedding object from ollama model:

```
embeddings = OllamaEmbeddings(model="mxbai-embed-large")
```

create vector database:

```
vector_store = Chroma(
    collection_name=collectionName,    # collection name
    persist_directory=dbFile,          # database file
    embedding_function=embeddings      # above embedding object
)
```

This function gives us the endpoint for the vector database. If the database does not exists then it will create the vector database. Then I'll create Document object for each row in the input *DataFrame* and add it to my ChromaDB instance.

```
# ...
docs = []
iids = []
for i, row in dfInput.iterrows(): # input dataframe with text data from sample.csv file
```

 # …

iids.append(str(i))

docs.append(Document(page_content=row["text"]))

 # …

vector_store.add_documents(documents=docs, ids=iids,…)

Query and Response workflow:

- Create retriever object based on similarity search. *retriever = Chroma(collection_name=collectionName, persist_directory=db_file).as_retriever(search_type="similarity", search_kwargs={"k": 5})*

  This creates a connection to a Chroma vector database, which stores your embedded documents and then converts the Chroma database into a retriever object that Ollama (or LangChain) can use to fetch relevant documents. search_type="similarity" tells the retriever to return documents based on vector similarity—i.e., the closest matches to the query embedding. Optionally, we can specify that the retriever should return the top 5 most similar documents. So, this basically configures a retriever for use with our Ollama's RAG (Retrieval-Augmented Generation) workflow.
- *model = OllamaLLM(model="llama3.2")* - it creates a connection to the *llama3.2* model running in *Ollama*, allowing your Python code to send prompts and receive responses through *LangChain's LLM* interface.
- Prompt Template – It instructs the model to act like a customer-support expert following the template.

  template = """

  You are an expert in answering questions about Customer Support

Here are some relevant reviews: {reviews}

Here is the question to answer: {question}

"""

- It defines the structure of the message that will be sent to the *LLM* (in my case, *Ollama* running *llama3.2*).
- Instruction to the model: "You are an expert in answering questions about Customer Support". This sets the model's role and domain expertise.
- *{reviews}* placeholder: This will be dynamically filled with retrieved context—usually customer reviews or support logs fetched by your retriever. It's part of your RAG pipeline.
- *{question}* placeholder: This is the user's actual question. The model will use both the question and the retrieved reviews to generate an answer.

- Main Q-A execution loop:

```
while True:
    print("\n\n-------------------------------")
    question = input("Ask your question (q to quit): ")
    print("\n\n")
    if question == "q":
        break
    reviews = retriever.invoke(question)
    result = chain.invoke({"reviews": reviews, "question": question})
    print(result)
```

Full implementation files: *csc525_module8_portfolio_project.py* and *persistChromaDb_Utils.py*, I've added multiple inline comments in these files and uploaded with the submission. Please review this uploaded, properly formatted Python implementation files. The source code file also has some more helper functions for the supporting operations. This is also uploaded at my GitHub: *https://github.com/csu-mm/CSC525*.

Environment Setup and Installs:

```
#
# (customer support logs) input data file: sample.csv
# used data columns: tweet_id, created_at, text
# dataset available here
#
# Environment, Installs
# My dev environment: Windows 11 Enterprise, GPU, VS Code, Python 3.13.5
# 1. Install Ollama services/runtimes: download and install from: https://ollama.com/
# 2. after installation, make sure ollama service running
#   (ping ollama API endpoint)  http://localhost:11434/api/version
#     -- it should not fail or show error or 404 or localhost refused to connect, etc.
# C:\Windows\System32>ollama list
#     -- above command will start the ollama services/runtimes, if it was not running.
#     -- show locally installed models.
#    (This may need the same user account that used during installation or administrator
          privileges.)
#
```

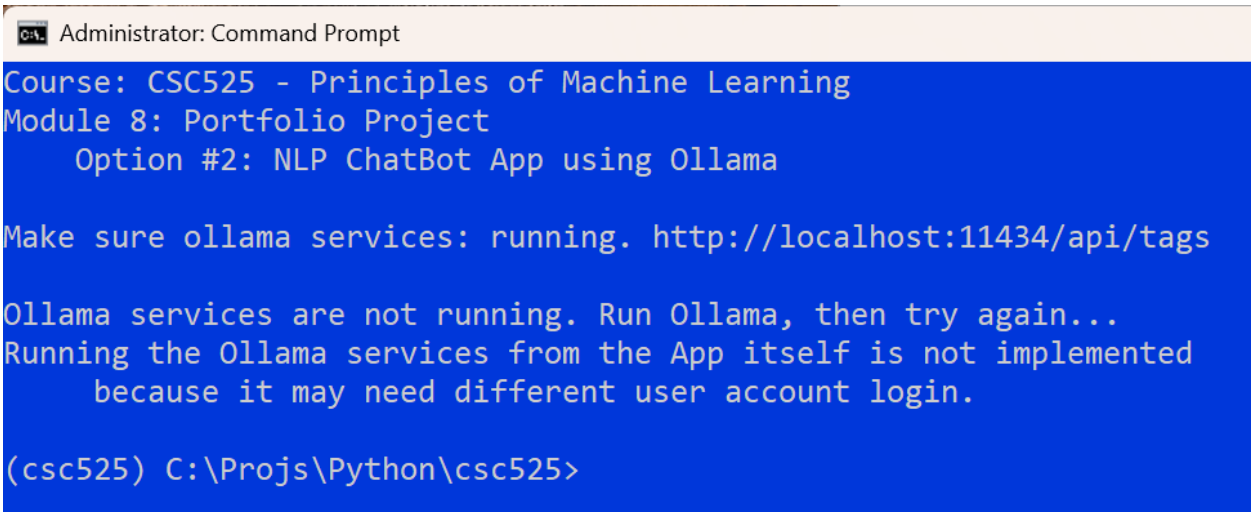
```
# 3. Install Ollama models:
#    3a. C:\Windows\System32>ollama pull llama3.2
#    3b. C:\Windows\System32>ollama pull mxbai-embed-large
# to check installed models, run: C:\Users\User1>ollama list
#   or
# (ping API endpoint) http://localhost:11434/api/tags
# Create virtual environment: C:\Projs\Python>python -m venv csc525
#Activate virtual environment: C:\Projs\Python\csc525>Scripts\activate
# --- needed pip installs ---
# (csc525) C:\Projs\Python\csc525>python.exe -m pip install --upgrade pip
# (csc525) C:\Projs\Python\csc525>pip install pandas requests
# (csc525) C:\Projs\Python\csc525>pip install langchain-core langchain langchain-ollama
# (csc525) C:\Projs\Python\csc525>pip install langchain-chroma
# (csc525) C:\Projs\Python\csc525>pip install --upgrade chromadb
    ( this is needed for: persistChromaDb_Utils.py )
#
# Above components needed to be installed.
# Followings are for test and execution purpose.
# to run a model: C:\Windows\System32>ollama run llama3.2
#
# If the model is updating/upgrading, it may show Error: upgrade in progress...
#
```

Chatbot App Execution and Results:

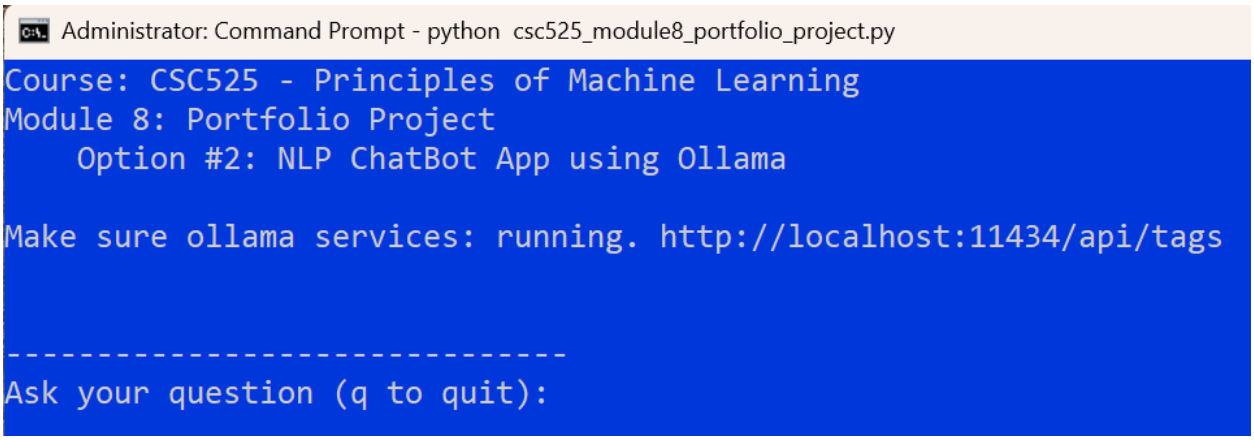
First run: *ollama* runtime/services: C:\Users\User1>ollama list

Without running Ollama, ChatBot app won't work. Then run ChatBot app

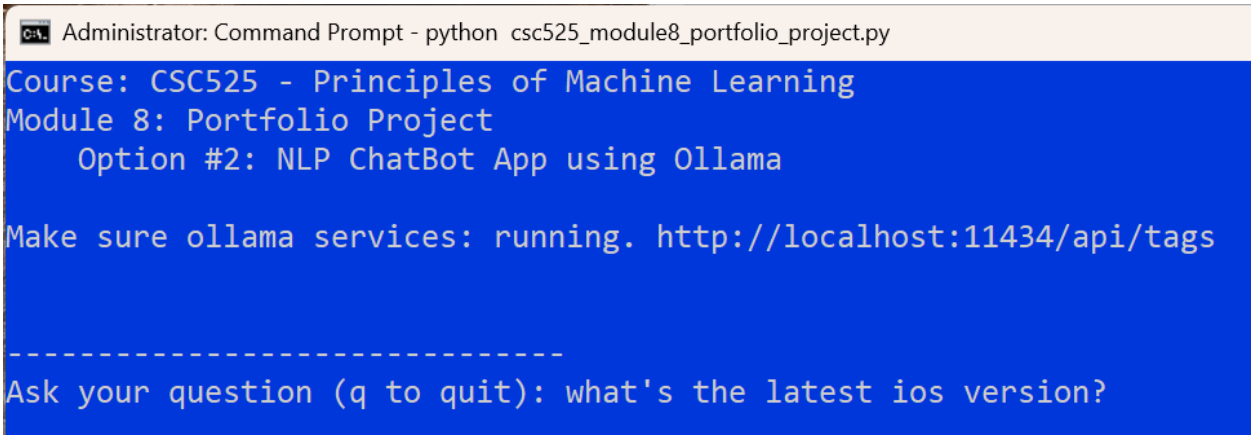
(csc525) C:\Projs\Python\csc525>python csc525_module8_portfolio_project.py



First screen (if Ollama is not running)

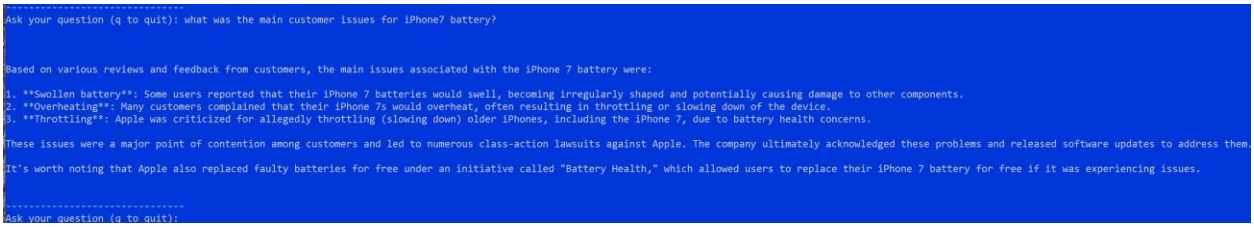


First screen (if Ollama is running)

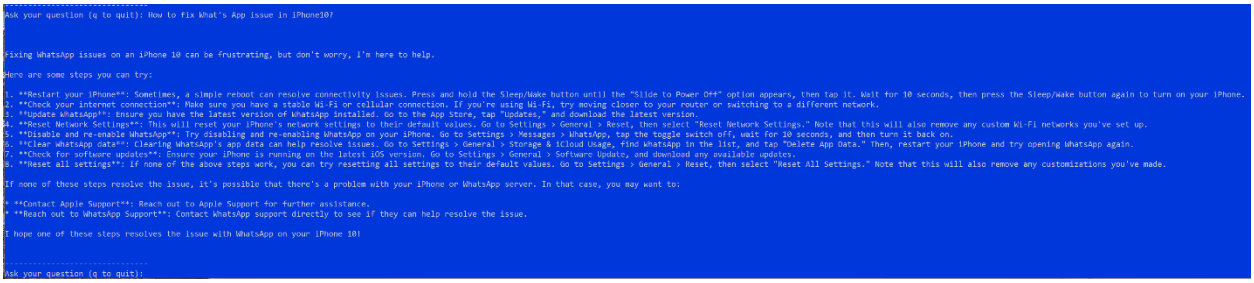


User asking question: What's the latest ios version? (local vector db should not have the data for this question, because used data is from old iPhone customer support logs)

Response to User (without data, logically acceptable response)



QA (with data in the local vector db)



QA (with data in the local vector db)

Whether this chatbot is open-domain or closed?

This chatbot is a closed-domain chatbot, not an open-domain one. Here the chatbot's knowledge is restricted; it only sees: system local data for customer support reviews, customer support logs, customer support instructions etc., nothing for other general knowledge. Also, the chatbot's prompt restricts its behavior and says: You are an expert in answering questions about Customer Support. This is a domain constraint, a closed domain. My chatbot app is closed-domain because it relies on a retrieval system built from your own customer-support reviews stored in Chroma. The embedding model mxbai-embed-large and the LLM llama3.2 only generate answers using that domain-specific context. Since the prompt instructs the model to act as a customer-support expert and the retriever supplies only customer-support data, the chatbot stays restricted to that domain rather than answering general open-domain questions. The chatbot is a customer-support specialist, not a general chat assistant. That makes it closed-domain.

Conclusion: This generative AI based chatbot app works successfully. Still to make it more popular or user friendly, I can provide these value-additions.

- By shifting from plain command-line exchanges to rich, intuitive web interfaces, I can make it more user-friendly generative AI app. A well-designed web UI offers visual structure, interactive elements, and clearer feedback, making complex AI capabilities feel accessible. Features like context panels, inline previews, and guided workflows help users understand responses instantly. This evolution transforms AI from a technical tool into a seamless, user-friendly experience that supports creativity and productivity.
- Another obvious improvement would be the use of a multi model RAG pipeline. A multi model RAG pipeline strengthens GenAI apps by combining retrieval, reasoning, and

generation into one adaptive workflow. It blends vector search, semantic reranking, and specialized models to deliver precise, context aware outputs. Each model contributes strengths such as classification, summarization, or generation, while retrieval ensures grounding in trusted data. This layered design boosts accuracy, reduces hallucinations, and supports scalable, reliable AI applications.

References

DevTutorial. (n.d.). Local AI with Ollama: Building a RAG system with Ollama and LangChain. https://devtutorial.io/local-ai-with-ollama-building-a-rag-system-with-ollama-and-langchain-p3820.html

Embedding (machine learning). (n.d.). Wikipedia. https://en.wikipedia.org/wiki/Embedding_(machine_learning)

Ollama. (n.d.). Documentation. https://docs.ollama.com/

Retrieval-augmented generation. (n.d.). Wikipedia. https://en.wikipedia.org/wiki/Retrieval-augmented_generation

Shamim, W. (2025, February 21). Building a local RAG-based chatbot using ChromaDB, LangChain, and Streamlit and Ollama. Medium. https://medium.com/@Shamimw/building-a-local-rag-based-chatbot-using-chromadb-langchain-and-streamlit-and-ollama-9410559c8a4d

Vector database. (n.d.). Wikipedia. https://en.wikipedia.org/wiki/Vector_database